



POLITECNICO
MILANO 1863

Search-Based Test Generation for Autonomous UAV Obstacle Avoidance Using a $(1 + 1)$ -Evolution Strategy

Software Engineering for Automation – Project Proposal

Authors: Roberto Capone and Guglielmo Cattaneo **Instructor:** Prof. Matteo Camilli
Politecnico di Milano – A.Y. 2025–2026 **Release date:** 17/07/2026 **Version:** 1.0

Context and problem. Autonomous unmanned aerial vehicles are safety-critical cyber-physical systems whose obstacle-avoidance behaviour is difficult and expensive to test in the physical world. The UAV Testing Competition provides PX4-Avoidance as the system under test and Aerialist as a simulation-based test bench. A test generator must produce valid obstacle configurations that expose unsafe trajectories or collisions while respecting a bounded simulation budget and the competition constraints.

Project objective. The project develops a robust, reproducible, and modular test-generation tool based on a $(1 + 1)$ -Evolution Strategy. The search minimises the minimum distance between the UAV trajectory and the generated obstacles. The tool additionally preserves diverse failures, favours simple test cases in the final ranking, mitigates simulator non-determinism, and recovers from simulator or container failures without losing the search state.

Scope. The system accepts an Aerialist YAML case study and a simulation budget, generates one to three legal non-overlapping box obstacles, executes candidate tests through Aerialist and PX4/Gazebo, evaluates the observed trajectories, maintains an archive of failing tests, and exports an ordered delivery suite. The project includes a random-search baseline for experimental comparison. PX4, Gazebo, Aerialist, the mission definitions, and the official evaluator are external systems and are not modified.

Principal functions.

- adaptive obstacle sampling along the nominal mission route, with route bootstrapping when no nominal flight log is available;
- Gaussian mutation, Rechenberg's $1/5$ success rule, stagnation detection, diversified restart, and targeted intensification;
- simulation execution with timeout, retries, budget accounting, and optional confirmation of highly critical candidates;
- persistent checkpointing and exact re-execution of an interrupted candidate after a crash;
- failure archiving, trajectory-space diversity filtering through Dynamic Time Warping, official-style ranking, and generation of YAML, ULog, and plot artifacts;
- interchangeable execution of the evolutionary generator and an independent random-search baseline.

Quality objectives. The implementation shall prioritise correctness, compliance with obstacle constraints, recoverability, reproducibility, maintainability, traceability, and ease of execution in Docker. A complete run shall never exceed its declared simulation budget, and every delivered test shall be reconstructible from persistent artifacts.

Expected deliverables.

1. Requirements Analysis and Specification Document;
2. Design Document with structural, behavioural, deployment, and traceability views;
3. final implementation, execution instructions, UML models, tests, experiment outputs, and presentation in the project Git repository.

Reference implementation. <https://github.com/gulicatta/UAV-Testing-Competition/tree/es-generator>